

Cadastro e controle de produtos

Aluno: Carlos Eduardo Silva Ferreira

Matrícula: 202408542283

Professor: Alessandro Calin

Testes do menu e arquivo criado

Registro de execução - 01_cadastro_mouse

```
PS C:\programacao_software_basico_c\trabalho> .\produtos.exe
===== CONTROLE DE PRODUTOS =====
1. Cadastrar produto
2. Listar produtos
3. Buscar produto
4. Calcular valor do estoque
5. Atualizar quantidade
6. Sair
Escolha uma opcao: 1
=== Cadastro de produto ===
Codigo: 1001
Nome: Mouse sem fio
Categoria: Perifericos
Quantidade em estoque: 2
Preco unitario: R$ 149.90
Produto cadastrado com sucesso.
===== CONTROLE DE PRODUTOS =====
1. Cadastrar produto
2. Listar produtos
3. Buscar produto
4. Calcular valor do estoque
5. Atualizar quantidade
6. Sair
Escolha uma opcao: 6
Programa encerrado.
```

Registro de execução - 02_cadastro_teclado

```
PS C:\programacao_software_basico_c\trabalho> .\produtos.exe
===== CONTROLE DE PRODUTOS =====
1. Cadastrar produto
2. Listar produtos
3. Buscar produto
4. Calcular valor do estoque
5. Atualizar quantidade
6. Sair
Escolha uma opcao: 1
=== Cadastro de produto ===
Codigo: 1002
Nome: Teclado mecanico
Categoria: Perifericos
Quantidade em estoque: 5
Preco unitario: R$ 25,50
Produto cadastrado com sucesso.
===== CONTROLE DE PRODUTOS =====
1. Cadastrar produto
2. Listar produtos
3. Buscar produto
4. Calcular valor do estoque
5. Atualizar quantidade
6. Sair
Escolha uma opcao: 6
Programa encerrado.
```

Registro de execução - 03_listar_produtos

```
PS C:\programacao_software_basico_c\trabalho> .\produtos.exe
===== CONTROLE DE PRODUTOS =====
1. Cadastrar produto
2. Listar produtos
3. Buscar produto
4. Calcular valor do estoque
5. Atualizar quantidade
6. Sair
Escolha uma opcao: 2
=== Produtos cadastrados ===
Codigo: 1001 | Nome: Mouse sem fio | Categoria: Perifericos | Quantidade: 2 | Preço: R$ 149.90
Codigo: 1002 | Nome: Teclado mecanico | Categoria: Perifericos | Quantidade: 5 | Preço: R$ 25.50
===== CONTROLE DE PRODUTOS =====
1. Cadastrar produto
2. Listar produtos
3. Buscar produto
4. Calcular valor do estoque
5. Atualizar quantidade
6. Sair
Escolha uma opcao: 6
Programa encerrado.
```

Registro de execução - 04_busca_encontrado

```
PS C:\programacao_software_basico_c\trabalho> .\produtos.exe
===== CONTROLE DE PRODUTOS =====
1. Cadastrar produto
2. Listar produtos
3. Buscar produto
4. Calcular valor do estoque
5. Atualizar quantidade
6. Sair
Escolha uma opcao: 3
=== Busca de produto ===
Codigo para buscar: 1001
Produto encontrado:
Codigo: 1001 | Nome: Mouse sem fio | Categoria: Perifericos | Quantidade: 2 | Preço: R$ 149.90
===== CONTROLE DE PRODUTOS =====
1. Cadastrar produto
2. Listar produtos
3. Buscar produto
4. Calcular valor do estoque
5. Atualizar quantidade
6. Sair
Escolha uma opcao: 6
Programa encerrado.
```

Registro de execução - 05_busca_nao_encontrado

```
PS C:\programacao_software_basico_c\trabalho> .\produtos.exe
===== CONTROLE DE PRODUTOS =====
1. Cadastrar produto
2. Listar produtos
3. Buscar produto
4. Calcular valor do estoque
5. Atualizar quantidade
6. Sair
Escolha uma opcao: 3
=== Busca de produto ===
Codigo para buscar: 9999
Produto com codigo 9999 nao encontrado.
===== CONTROLE DE PRODUTOS =====
1. Cadastrar produto
2. Listar produtos
3. Buscar produto
4. Calcular valor do estoque
5. Atualizar quantidade
6. Sair
Escolha uma opcao: 6
Programa encerrado.
```

Registro de execução - 06_total_antes_atualizacao

```
PS C:\programacao_software_basico_c\trabalho> .\produtos.exe
===== CONTROLE DE PRODUTOS =====
1. Cadastrar produto
2. Listar produtos
3. Buscar produto
4. Calcular valor do estoque
5. Atualizar quantidade
6. Sair
Escolha uma opcao: 4
=== Valor total do estoque ===
Valor total do estoque: R$ 427,30
===== CONTROLE DE PRODUTOS =====
1. Cadastrar produto
2. Listar produtos
3. Buscar produto
4. Calcular valor do estoque
5. Atualizar quantidade
6. Sair
Escolha uma opcao: 6
Programa encerrado.
```

Registro de execução - 07_atualizar_quantidade

```
PS C:\programacao_software_basico_c\trabalho> .\produtos.exe
===== CONTROLE DE PRODUTOS =====
1. Cadastrar produto
2. Listar produtos
3. Buscar produto
4. Calcular valor do estoque
5. Atualizar quantidade
6. Sair
Escolha uma opcao: 5
=== Atualizar quantidade em estoque ===
Codigo do produto: 1001
Nova quantidade: 4
Quantidade atualizada com sucesso.
===== CONTROLE DE PRODUTOS =====
1. Cadastrar produto
2. Listar produtos
3. Buscar produto
4. Calcular valor do estoque
5. Atualizar quantidade
6. Sair
Escolha uma opcao: 6
Programa encerrado.
```

Registro de execução - 08_listar_apos_atualizacao

```
PS C:\programacao_software_basico_c\trabalho> .\produtos.exe
===== CONTROLE DE PRODUTOS =====
1. Cadastrar produto
2. Listar produtos
3. Buscar produto
4. Calcular valor do estoque
5. Atualizar quantidade
6. Sair
Escolha uma opcao: 2
=== Produtos cadastrados ===
Codigo: 1001 | Nome: Mouse sem fio | Categoria: Perifericos | Quantidade: 4 | Preço: R$ 149.90
Codigo: 1002 | Nome: Teclado mecanico | Categoria: Perifericos | Quantidade: 5 | Preço: R$ 25.50
===== CONTROLE DE PRODUTOS =====
1. Cadastrar produto
2. Listar produtos
3. Buscar produto
4. Calcular valor do estoque
5. Atualizar quantidade
6. Sair
Escolha uma opcao: 6
Programa encerrado.
```

Registro de execução - 09_total_apos_atualizacao

```
PS C:\programacao_software_basico_c\trabalho> .\produtos.exe
===== CONTROLE DE PRODUTOS =====
1. Cadastrar produto
2. Listar produtos
3. Buscar produto
4. Calcular valor do estoque
5. Atualizar quantidade
6. Sair
Escolha uma opcao: 4
=== Valor total do estoque ===
Valor total do estoque: R$ 727.10
===== CONTROLE DE PRODUTOS =====
1. Cadastrar produto
2. Listar produtos
3. Buscar produto
4. Calcular valor do estoque
5. Atualizar quantidade
6. Sair
Escolha uma opcao: 6
Programa encerrado.
```

Registro de execução - 10_sair

```
PS C:\programacao_software_basico_c\trabalho> .\produtos.exe
===== CONTROLE DE PRODUTOS =====
1. Cadastrar produto
2. Listar produtos
3. Buscar produto
4. Calcular valor do estoque
5. Atualizar quantidade
6. Sair
Escolha uma opcao: 6
Programa encerrado.
```

Arquivo criado - produtos.txt

```
PS C:\programacao_software_basico_c\trabalho> Get-Content .\produtos.txt
1001|Mouse sem fio|Perifericos|4|149.90
1002|Teclado mecanico|Perifericos|5|25.50
```

Código-fonte completo

Arquivo: produtos.c

```
#include <ctype.h>
#include <errno.h>
#include <math.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define ARQUIVO_PRODUTOS "produtos.txt"
#define ARQUIVO_TEMPORARIO "produtos.tmp"
#define ARQUIVO_BACKUP "produtos.bak"
#define TAMANHO_NOME 80
#define TAMANHO_CATEGORIA 50
#define TAMANHO_LINHA 256

typedef struct {
    int codigo;
    char nome[TAMANHO_NOME];
    char categoria[TAMANHO_CATEGORIA];
    int quantidade;
    double preco;
} Produto;

static int ler_linha(const char *mensagem, char *destino, size_t tamanho) {
    size_t comprimento;

    printf("%s", mensagem);
    if (fgets(destino, (int)tamanho, stdin) == NULL) {
        return 0;
    }

    comprimento = strlen(destino);
    if (comprimento > 0 && destino[comprimento - 1] == '\n') {
        destino[--comprimento] = '\0';
    } else {
        int caractere;
        while ((caractere = getchar()) != '\n' && caractere != EOF) {
        }
        printf("Entrada muito longa. Tente novamente.\n");
        return -1;
    }

    if (comprimento > 0 && destino[comprimento - 1] == '\r') {
        destino[comprimento - 1] = '\0';
    }
    return 1;
}

static int ler_inteiro(const char *mensagem, int minimo, int *valor) {
    char linha[80];
    char *fim;
    long numero;
    int resultado;

    for (;;) {
        resultado = ler_linha(mensagem, linha, sizeof(linha));
        if (resultado == 0) {
            return 0;
        }
        if (resultado < 0 || linha[0] == '\0') {
            printf("Digite um numero inteiro valido.\n");
            continue;
        }

        errno = 0;
        numero = strtol(linha, &fim, 10);
        while (isspace((unsigned char)*fim)) {
            fim++;
        }
        if (errno == 0 && fim != linha && *fim == '\0' && numero >= minimo && numero <= 2147483647L) {
            *valor = (int)numero;
            return 1;
        }
        printf("Valor invalido. Digite um inteiro maior ou igual a %d.\n", minimo);
    }
}

static int ler_preco(const char *mensagem, double *valor) {
    char linha[80];
    char *fim;
    size_t indice;
    int resultado;
    double numero;

    for (;;) {
```

```

        resultado = ler_linha(mensagem, linha, sizeof(linha));
        if (resultado == 0) {
            return 0;
        }
        if (resultado < 0 || linha[0] == '\0') {
            printf("Digite um preco valido, por exemplo 129.90.\n");
            continue;
        }
        for (indice = 0; linha[indice] != '\0'; indice++) {
            if (linha[indice] == ',') {
                linha[indice] = '.';
            }
        }

        errno = 0;
        numero = strtod(linha, &fim);
        while (isspace((unsigned char)*fim)) {
            fim++;
        }
        if (errno == 0 && fim != linha && *fim == '\0' && isfinite(numero) && numero >= 0.0) {
            *valor = numero;
            return 1;
        }
        printf("Preco invalido. Use um numero maior ou igual a zero.\n");
    }
}

static int campo_valido(const char *texto) {
    return texto[0] != '\0' && strchr(texto, '|') == NULL;
}

static int ler_texto_produto(const char *mensagem, char *destino, size_t tamanho) {
    int resultado;
    do {
        resultado = ler_linha(mensagem, destino, tamanho);
        if (resultado == 0) {
            return 0;
        }
        if (resultado > 0 && !campo_valido(destino)) {
            printf("O campo nao pode ficar vazio nem conter o caractere |.\n");
            resultado = -1;
        }
    } while (resultado < 0);
    return 1;
}

static int ler_produto(FILE *arquivo, Produto *produto) {
    char linha[TAMANHO_LINHA];
    char extra;
    int lidos;

    if (fgets(linha, sizeof(linha), arquivo) == NULL) {
        return 0;
    }
    if (strchr(linha, '\n') == NULL && !feof(arquivo)) {
        int caractere;
        while ((caractere = fgetc(arquivo)) != '\n' && caractere != EOF) {
        }
        return -1;
    }

    lidos = sscanf(linha, "%d|%79[^\n]|%49[^\n]|%d|f %c",
        &produto->codigo, produto->nome, produto->categoria,
        &produto->quantidade, &produto->preco, &extra);
    if (lidos != 5 || produto->codigo <= 0 || produto->quantidade < 0 ||
        !isfinite(produto->preco) || produto->preco < 0.0) {
        return -1;
    }
    return 1;
}

static int codigo_existe(int codigo) {
    FILE *arquivo = fopen(ARQUIVO_PRODUTOS, "r");
    Produto produto;
    int resultado;

    if (arquivo == NULL) {
        return 0;
    }
    while ((resultado = ler_produto(arquivo, &produto)) != 0) {
        if (resultado > 0 && produto.codigo == codigo) {
            fclose(arquivo);
            return 1;
        }
    }
    fclose(arquivo);
    return 0;
}

static void cadastrar_produto(void) {

```

```

Produto produto;
FILE *arquivo;
int erro_escrita;
int erro_fechamento;

printf("\n=== Cadastro de produto ===\n");
if (!ler_inteiro("Codigo: ", 1, &produto.codigo) ||
    !ler_texto_produto("Nome: ", produto.nome, sizeof(produto.nome)) ||
    !ler_texto_produto("Categoria: ", produto.categoria, sizeof(produto.categoria)) ||
    !ler_inteiro("Quantidade em estoque: ", 0, &produto.quantidade) ||
    !ler_preco("Preco unitario: R$ ", &produto.preco)) {
    printf("Cadastro cancelado.\n");
    return;
}

if (codigo_existe(produto.codigo)) {
    printf("Ja existe um produto com esse codigo.\n");
    return;
}

arquivo = fopen(ARQUIVO_PRODUTOS, "a");
if (arquivo == NULL) {
    printf("Nao foi possivel abrir o arquivo de produtos.\n");
    return;
}
erro_escrita = fprintf(arquivo, "%d|%s|%s|%d|%.2f\n", produto.codigo, produto.nome,
    produto.categoria, produto.quantidade, produto.preco) < 0;
erro_fechamento = fclose(arquivo) != 0;
if (erro_escrita || erro_fechamento) {
    printf("Erro ao gravar o produto no arquivo.\n");
    return;
}
printf("Produto cadastrado com sucesso.\n");
}

static void imprimir_produto(const Produto *produto) {
    printf("Codigo: %d | Nome: %s | Categoria: %s | Quantidade: %d | Preco: R$ %.2f\n",
        produto->codigo, produto->nome, produto->categoria,
        produto->quantidade, produto->preco);
}

static void listar_produtos(void) {
    FILE *arquivo = fopen(ARQUIVO_PRODUTOS, "r");
    Produto produto;
    int resultado;
    int encontrados = 0;

    printf("\n=== Produtos cadastrados ===\n");
    if (arquivo == NULL) {
        printf("Nenhum produto cadastrado.\n");
        return;
    }
    while ((resultado = ler_produto(arquivo, &produto)) != 0) {
        if (resultado > 0) {
            imprimir_produto(&produto);
            encontrados++;
        } else {
            printf("Aviso: uma linha invalida foi ignorada no arquivo.\n");
        }
    }
    if (ferror(arquivo)) {
        printf("Erro durante a leitura do arquivo.\n");
    } else if (encontrados == 0) {
        printf("Nenhum produto cadastrado.\n");
    }
    fclose(arquivo);
}

static void buscar_produto(void) {
    int codigo;
    FILE *arquivo;
    Produto produto;
    int resultado;

    printf("\n=== Busca de produto ===\n");
    if (!ler_inteiro("Codigo para buscar: ", 1, &codigo)) {
        printf("Busca cancelada.\n");
        return;
    }
    arquivo = fopen(ARQUIVO_PRODUTOS, "r");
    if (arquivo == NULL) {
        printf("Nenhum produto cadastrado.\n");
        return;
    }
    while ((resultado = ler_produto(arquivo, &produto)) != 0) {
        if (resultado > 0 && produto.codigo == codigo) {
            printf("Produto encontrado:\n");
            imprimir_produto(&produto);
            fclose(arquivo);
            return;
        }
    }
}

```

```

    }
}
fclose(arquivo);
printf("Produto com codigo %d nao encontrado.\n", codigo);
}

static void calcular_valor_estoque(void) {
    FILE *arquivo = fopen(ARQUIVO_PRODUTOS, "r");
    Produto produto;
    double total = 0.0;
    int resultado;
    int encontrados = 0;

    printf("\n=== Valor total do estoque ===\n");
    if (arquivo == NULL) {
        printf("Nenhum produto cadastrado. Valor total: R$ 0.00\n");
        return;
    }
    while ((resultado = ler_produto(arquivo, &produto)) != 0) {
        if (resultado > 0) {
            total += produto.quantidade * produto.preco;
            encontrados++;
        }
    }
    fclose(arquivo);
    if (encontrados == 0) {
        printf("Nenhum produto cadastrado. ");
    }
    printf("Valor total do estoque: R$ %.2f\n", total);
}

static int gravar_produto(FILE *arquivo, const Produto *produto) {
    return fprintf(arquivo, "%d|s|s|d|%.2f\n", produto->codigo,
        produto->nome, produto->categoria,
        produto->quantidade, produto->preco) >= 0;
}

static void atualizar_quantidade(void) {
    int codigo;
    int nova_quantidade;
    int encontrado = 0;
    FILE *original;
    FILE *temporario;
    Produto produto;
    int resultado;
    int erro_leitura;
    int erro_fechamento_original;
    int erro_fechamento_temporario;

    printf("\n=== Atualizar quantidade em estoque ===\n");
    if (!ler_inteiro("Codigo do produto: ", 1, &codigo) ||
        !ler_inteiro("Nova quantidade: ", 0, &nova_quantidade)) {
        printf("Atualizacao cancelada.\n");
        return;
    }
    original = fopen(ARQUIVO_PRODUTOS, "r");
    if (original == NULL) {
        printf("Nenhum produto cadastrado.\n");
        return;
    }
    temporario = fopen(ARQUIVO_TEMPORARIO, "w");
    if (temporario == NULL) {
        fclose(original);
        printf("Nao foi possivel criar o arquivo temporario.\n");
        return;
    }

    while ((resultado = ler_produto(original, &produto)) != 0) {
        if (resultado < 0) {
            fclose(original);
            fclose(temporario);
            remove(ARQUIVO_TEMPORARIO);
            printf("Arquivo de produtos contem uma linha invalida; atualizacao cancelada.\n");
            return;
        }
        if (produto.codigo == codigo) {
            produto.quantidade = nova_quantidade;
            encontrado = 1;
        }
        if (!gravar_produto(temporario, &produto)) {
            fclose(original);
            fclose(temporario);
            remove(ARQUIVO_TEMPORARIO);
            printf("Erro ao gravar o arquivo temporario.\n");
            return;
        }
    }
    erro_leitura = ferror(original);
    erro_fechamento_original = fclose(original) != 0;
    erro_fechamento_temporario = fclose(temporario) != 0;
}

```

```

    if (erro_leitura || erro_fechamento_original || erro_fechamento_temporario) {
        remove(ARQUIVO_TEMPORARIO);
        printf("Erro ao ler ou fechar os arquivos.\n");
        return;
    }
    if (!encontrado) {
        remove(ARQUIVO_TEMPORARIO);
        printf("Produto com codigo %d nao encontrado.\n", codigo);
        return;
    }

    remove(ARQUIVO_BACKUP);
    if (rename(ARQUIVO_PRODUTOS, ARQUIVO_BACKUP) != 0) {
        remove(ARQUIVO_TEMPORARIO);
        printf("Nao foi possivel proteger o arquivo original.\n");
        return;
    }
    if (rename(ARQUIVO_TEMPORARIO, ARQUIVO_PRODUTOS) != 0) {
        rename(ARQUIVO_BACKUP, ARQUIVO_PRODUTOS);
        remove(ARQUIVO_TEMPORARIO);
        printf("Falha ao substituir o arquivo; o original foi restaurado.\n");
        return;
    }
    remove(ARQUIVO_BACKUP);
    printf("Quantidade atualizada com sucesso.\n");
}

static void exibir_menu(void) {
    printf("\n===== CONTROLE DE PRODUTOS =====\n");
    printf("1. Cadastrar produto\n");
    printf("2. Listar produtos\n");
    printf("3. Buscar produto\n");
    printf("4. Calcular valor do estoque\n");
    printf("5. Atualizar quantidade\n");
    printf("6. Sair\n");
}

int main(void) {
    int opcao;

    for (;;) {
        exibir_menu();
        if (!ler_inteiro("Escolha uma opcao: ", 1, &opcao)) {
            printf("\nPrograma encerrado.\n");
            return 0;
        }
        switch (opcao) {
            case 1:
                cadastrar_produto();
                break;
            case 2:
                listar_produtos();
                break;
            case 3:
                buscar_produto();
                break;
            case 4:
                calcular_valor_estoque();
                break;
            case 5:
                atualizar_quantidade();
                break;
            case 6:
                printf("Programa encerrado.\n");
                return 0;
            default:
                printf("Opcao invalida. Escolha uma opcao de 1 a 6.\n");
        }
    }
}

```